

Bug 修复日志 — 谢真婷（测试保障）

Bug #1: ApiCode 错误码冲突 — 4003 被两个不同语义复用

发现时间：2026-05-18

发现阶段：单元测试设计前的代码审查

现象

ApiCode 枚举中 USERNAME_TAKEN 定义为 4003，但 EvaluationServiceImpl.submitEvaluation() 中直接使用硬编码 new BusinessException(4003, "订单尚未完成，暂不可提交评价")，导致错误码 4003 同时表示"用户名已存在"和"订单未完成"两种完全不相关的业务错误。

根因

ApiCode 枚举的错误码规划不够充分，未覆盖评价模块的业务错误码；评价模块实现时直接硬编码了 4003 而非在 ApiCode 中新增枚举值。

涉及文件

- backend/src/main/java/com/campushub/common/exception/ApiCode.java — 缺少评价相关错误码
- backend/src/main/java/com/campushub/evaluation/service/impl/EvaluationServiceImpl.java:51 — 硬编码 4003

修复方案

在 ApiCode 中新增评价相关错误码，并修改 EvaluationServiceImpl 引用枚举而非硬编码。

验证结果

已在集成测试中复现确认。FullFlowIntegrationTest 中第三方评价测试（订单IN_PROGRESS状态）实际返回 4003（订单未完成）而非 4004（非参与者），因为 EvaluationServiceImpl 的校验顺序是先检查订单状态再检查参与者身份。测试已更新为覆盖两种场景。

建议：在 ApiCode 中新增 ORDER_NOT_COMPLETED 和 NOT_ORDER_PARTICIPANT 等评价专属错误码，清理 4003 的语义冲突。

Bug #2: NotificationService 使用时间戳作为主键，高并发下可能碰撞 ✓ 已修复

发现时间：2026-05-18

发现阶段：单元测试设计前的代码审查

现象

`NotificationService.createNotification()` 使用 `LocalDateTime.now().toEpochMilli()` 作为 `notification_id`。在同一毫秒内创建多条通知时（如同一订单状态变更同时通知发布者和接单者），会产生主键冲突导致第二条插入失败。

根因

`notification_id` 生成策略使用了毫秒级时间戳，精度不足以在亚毫秒间隔内保证唯一性。其他实体类使用 MyBatis-Plus 的 `IdType.ASSIGN_ID`（雪花算法）生成 ID，但 `NotificationService` 使用纯 JDBC 因此无法享受雪花 ID。

涉及文件

- `backend/src/main/java/com/campushub/notification/NotificationService.java:77-80`

修复方案

将 `notification_id` 生成从 `LocalDateTime.now().toEpochMilli()` 改为 `AtomicLong.incrementAndGet()`（起始值为 `System.currentTimeMillis()`），确保单 JVM 内 ID 唯一。

验证结果

修复前：`OrderServiceTest`（20个测试中 1 个 error）、`FullFlowIntegrationTest`（6个测试中 2 个 error）、`NotificationServiceTest`（13个测试中 3 个 error）均因主键冲突失败。

修复后：全部 79 个测试通过。`createNotification_MultipleForSameUser_ShouldAllPersist` 在同一毫秒内创建 3 条通知，修复后无碰撞。

Bug #3: EvaluationServiceImpl 测试数据未覆盖所有星级和异常场景

发现时间：2026-05-18

发现阶段：测试用例设计

现象

现有 EvaluationServiceImplTest 仅有 3 个测试用例（5 星加分、1 星扣分保底、订单未完成拒绝），缺少 4 星/3 星/2 星的信用分计算验证，也缺少"非参与者评价"和"重复评价"的异常路径。

涉及文件

- backend/src/test/java/com/campushub/evaluation/service/EvaluationServiceImplTest.java

修复方案

已在测试设计中补充全部星级测试 + 异常路径测试（见本 PR 测试代码）。

验证结果

补充后测试用例从 3 个增加到 10 个，覆盖全部星级计算和所有异常路径。

2026-05-19 新增：系统集成联调发现并修复的 Bug

Bug #4: 需求列表分页 — total 被前端覆盖导致翻页异常 ✓ 已修复

发现时间： 2026-05-19 | 发现阶段： 系统集成联调（任务 5）

现象： 需求列表存在 CANCELED（已取消）状态需求时，分页"下一页"按钮被异常禁用，后端返回了正确的 total 和更多数据。

根因： RequirementListView.vue:112 中 total.value = requirements.value.length 使用客户端过滤后的数组长度覆盖了后端 data.total。activeRequirements() 过滤 CANCELED 项后数组变短，导致 page * pageSize >= total 过早为 true。同样问题在 loadRecommendations()（第 136 行）也存在。

涉及文件： frontend/src/views/RequirementListView.vue:112,136

修复方案： 两处 total.value = requirements.value.length → total.value = data.total

验证结果： 修复后前端构建成功，分页使用后端真实 total。

Bug #5: 筛选下拉包含无效类型 LOST_FOUND ✓ 已修复

发现时间： 2026-05-19 | **发现阶段：** 系统集成联调（任务 5）

现象： 需求大厅类型筛选选择"失物招领"（LOST_FOUND）后永远返回 0 条结果。

根因： 前端筛选下拉硬编码了 LOST_FOUND，后端 RequirementType 枚举无此值。同时筛选下拉缺少 5 种有效类型。

涉及文件： frontend/src/views/RequirementListView.vue:13-21

修复方案： 删除 LOST_FOUND，补充缺失的 STUDY_HELP / MATERIAL_SHARE / CARPOOL / Q_AND_A / OTHER，统一 TUTORING 标签为"学业辅导"。

验证结果： 筛选下拉 9 个选项与后端枚举 9 个值完全一致。

Bug #6: typeLabel 函数枚举名与后端不一致 ✓ 已修复

发现时间： 2026-05-19 | **发现阶段：** 系统集成联调（任务 5）

现象： STUDY_HELP 等类型在前端显示为英文枚举名，无法映射为中文标签。

根因： 两个 Vue 文件中的 typeLabel() 函数 key 值与后端 RequirementType.java 不匹配：STUDY → 应为 STUDY_HELP，MATERIAL → 应为 MATERIAL_SHARE，QA → 应为 Q_AND_A，LOST_FOUND 不存在。

涉及文件：

- frontend/src/views/RequirementListView.vue:92-104
- frontend/src/views/RequirementDetailView.vue:70-82

修复方案： 修正两处 typeLabel 函数 key 值，删除 LOST_FOUND。

验证结果： 修复后前端构建成功，与后端枚举完全对齐。